

Deployment & Infrastructure Pipeline Specification

Project: NGO Platform | Architecture: Decoupled Nuxt & Node.js

1. Infrastructure Stack Matrix

The platform relies on a distributed deployment model to achieve cost-efficiency while maintaining high performance and clear separation of concerns.

Component	Provider	Role / Specifications
Domain DNS	Hostinger	Root domain registrar and DNS routing (A and CNAME records).
Frontend	Vercel / Netlify	Nuxt SSR hosting, automatic CI/CD from repository, SSL provisioning.
Backend API	Render	Node.js Web Service running the core business logic.
Database	Supabase	Persistent PostgreSQL instance managed externally.
Uptime Monitor	UptimeRobot	External cron job scheduler for infrastructure keep-alive.

2. Hexagonal Architecture & Structural Requirements

To ensure resilience against infrastructure shifts, the Node.js backend must strictly enforce a Hexagonal (Ports and Adapters) pattern.

- Domain Isolation:** The core NGO business logic must not contain any provider-specific code (e.g., Render-specific utilities or Supabase client libraries).
- Database Adapter:** The connection to Supabase must be encapsulated entirely within a dedicated PostgreSQL driven adapter. Inject the `DATABASE_URL` strictly at this boundary.
- Uptime Driving Port:** Implement a specific controller (driving adapter) mapped to the `/api/keep-alive` route. This port is explicitly designed to handle the external CRON pings without polluting standard user-facing routes.

3. The Uptime & Resource Optimization Protocol

Critical Infrastructure Constraints:

1. Render Web Services sleep after 15 minutes of inactivity.
2. Supabase Free Databases pause after 7 days of inactivity.

To bypass both constraints using a single mechanism without exceeding Render's 750 free monthly compute hours, the application requires an integrated keep-alive strategy.

Implementation details:

1. The external monitor (e.g., UptimeRobot) is configured to send an HTTP GET request to `https://api.yourngo.org/api/keep-alive` every 5 minutes.
2. The route's handler must invoke a lightweight database repository method.
3. The database adapter executes a minimal query: `SELECT 1 as healthy;` against Supabase.

This guarantees the Render instance remains warm (avoiding initial request latency) and registers continuous database activity in Supabase.

4. Step-by-Step Deployment Pipeline

Phase 1: Database Provisioning (Supabase)

- Create a new project in the Supabase dashboard.
- Navigate to Database settings and locate the Connection String (URI).
- Ensure the connection string uses the connection pooling URL (usually port `6543`) if available, as Serverless environments benefit from PgBouncer.
- **Output:** `SUPABASE_DB_URL` ready for injection.

Phase 2: Backend API Deployment (Render)

- Connect the Node.js repository to Render and create a new "Web Service".
- Set the Build Command (e.g., `npm ci`) and Start Command (e.g., `node dist/main.js`).
- In the Environment Variables section, securely add the `SUPABASE_DB_URL` mapping it to your application's expected variable name.
- Configure CORS in the Node.js app to temporarily accept requests from ``*`` or `localhost` during setup, pending final domain assignment.
- **Output:** A functional Render URL (e.g., `https://ngo-api-xyz.onrender.com`).

Phase 3: Frontend Deployment (Vercel/Netlify)

- Connect the Nuxt repository. The provider will auto-detect the framework and configure build steps.
- Inject the backend Render URL into the environment variables as `NEXT_PUBLIC_API_BASE`.

- Deploy the application.
- **Output:** A functional frontend URL.

Phase 4: Uptime Monitor Configuration

- Create a free account on UptimeRobot or Cron-job.org.
- Create a new HTTP GET monitor pointing to the Render URL:
`https://ngo-api-xyz.onrender.com/api/keep-alive` .
- Set the interval to 5 or 10 minutes.

Phase 5: DNS & SSL Routing (Hostinger)

Once both services are stable on their temporary URLs, finalize the production domains.

- **Frontend Routing:** Log into Hostinger hPanel. Point the root domain (e.g., `yourngo.org`) to Vercel/Netlify via A-records or an ALIAS/CNAME as per the provider's instructions.
- **Backend Routing:** Create a CNAME record for an API subdomain (e.g., `api.yourngo.org`) and point it to the Render temporary URL.
- **Finalizing CORS:** Update the Node.js backend configuration to restrict CORS exclusively to `https://yourngo.org` .

Security Check: Ensure that no environment variables (especially the Supabase database connection string) are committed to version control. They must only exist within the respective deployment provider's secure secret management dashboards.